



K.N. Toosi University of Technology

OFFLINE JAVA GUIDE

Chapter 6

Exceptions & Generics

How to handle runtime errors gracefully with try-catch-finally, and how to write type-safe reusable code using Java's generic types.

COURSE

Advanced Programming

SEMESTER

Spring 2026

DEPARTMENT

Computer Engineering

v1.0

Table of Contents

6.1	What is an Exception?	Errors vs exceptions, hierarchy
6.2	try-catch-finally	Catching and recovering
6.3	Multiple Catch & Hierarchy	Ordering, multi-catch
6.4	throw, throws & Custom Exceptions	Signaling and declaring
6.5	try-with-resources	Auto-closing resources
6.6	What Are Generics?	Type-safe collections
6.7	Generic Classes & Methods	Writing your own
6.8	Bounded Types & Wildcards	extends, super, ?
6.9	Common Mistakes & Tips	Pitfalls every beginner hits

6.1

What is an Exception?

An **exception** is an event that disrupts the normal flow of a program. Instead of crashing silently, Java **throws an exception object** that carries information about what went wrong and where.

Errors vs Exceptions

FEATURE	ERROR	EXCEPTION
Recoverable?	No — program should not try to catch these	Yes — program can and should handle these
Examples	<code>OutOfMemoryError</code> , <code>StackOverflowError</code>	<code>NullPointerException</code> , <code>FileNotFoundException</code>
Cause	Serious system-level problems	Logic problems, bad input, missing files
Inherits from	<code>Error</code>	<code>Exception</code>

The Exception Hierarchy

```
1  Throwable
2  |— Error           // don't catch these
3  |   |— OutOfMemoryError
4  |   |— StackOverflowError
5  |— Exception       // catch these
6  |   |— RuntimeException // unchecked - compiler doesn't force you to handle
7  |   |   |— NullPointerException
8  |   |   |— ArrayIndexOutOfBoundsException
9  |   |   |— NumberFormatException
10 |   |   |— ClassCastException
11 |   |   |— IllegalArgumentException
12 |   |   |— ArithmeticException
13 |   |— IOException   // checked - compiler REQUIRES you to handle
14 |       |— FileNotFoundException
15 |       |— EOFException
```

JAVA

Checked vs Unchecked

FEATURE	CHECKED	UNCHECKED
Compiler enforces handling?	Yes — must catch or declare	No — optional to handle
Extends	<code>Exception</code> (not <code>RuntimeException</code>)	<code>RuntimeException</code>
Typical cause	External factors (file missing, network down)	Programming bugs (null ref, bad index)
Examples	<code>IOException</code> , <code>FileNotFoundException</code>	<code>NullPointerException</code> , <code>ArithmeticException</code>

NOTE

The distinction matters at **compile time**. If you call a method that throws a checked exception and you don't handle it, your code **will not compile**. Unchecked exceptions compile fine — they only appear at runtime if triggered.

Basic Structure

```
1  try {
2      // Code that might throw an exception
3      int result = 10 / 0;
4  } catch (ArithmeticException e) {
5      // What to do if the exception occurs
6      System.out.println("Cannot divide by zero!");
7  } finally {
8      // Always runs – whether exception occurred or not
9      System.out.println("This always executes.");
10 }
```

JAVA

```
Cannot divide by zero! This always executes.
```

OUTPUT

How It Works

1. **try** — Attempt the risky code. If it succeeds, skip catch.
2. **catch** — If an exception of the specified type occurs, execute this block.
3. **finally** — Always runs, exception or not. Even if there's a `return` in try or catch.

Getting the Exception Message

```
1  try {
2      int[] arr = new int[3];
3      arr[5] = 10;
4  } catch (ArrayIndexOutOfBoundsException e) {
5      System.out.println("Message: " + e.getMessage());
6      // Message: Index 5 out of bounds for length 3
7  }
```

JAVA

What finally Is Really For

```
1  Scanner sc = null;
2  try {
3      sc = new Scanner(new File("data.txt"));
4      // ... use scanner ...
5  } catch (FileNotFoundException e) {
6      System.out.println("File not found: " + e.getMessage());
7  } finally {
8      if (sc != null) {
9          sc.close(); // always close the resource, even if an error occurred
10     }
11 }
```

JAVA

TIP

In practice, use **try-with-resources** (section 6.5) instead of `finally` for closing resources. It's shorter, safer, and guarantees cleanup even if you forget. The `finally` block is still useful for non-resource cleanup (logging, resetting state).

finally Edge Case

```
1  public static int test() {
2      try {
3          return 1;
4      } finally {
5          System.out.println("finally runs even with return!");
6      }
7  }
8  // test() returns 1, but "finally runs even with return!" prints first
```

JAVA

WARNING

If both `catch` and `finally` throw exceptions, the `finally` exception is the one that propagates up. Avoid throwing exceptions inside `finally`.

6.3 Multiple Catch & Hierarchy

Multiple catch Blocks

Chain multiple catch blocks to handle different exception types differently:

```
1  try {
2      String s = "hello";
3      int num = Integer.parseInt(s);
4      String[] arr = new String[2];
5      arr[5] = "x";
6  } catch (NumberFormatException e) {
7      System.out.println("Not a valid number: " + e.getMessage());
8  } catch (ArrayIndexOutOfBoundsException e) {
9      System.out.println("Index out of bounds: " + e.getMessage());
10 } catch (Exception e) {
11     System.out.println("Unexpected error: " + e.getMessage());
12 }
```

JAVA

Not a valid number: For input string: "hello"

OUTPUT

Catch Order Matters — Most Specific First

Catch blocks are checked **top to bottom**. The first matching type wins. If you put a parent type before a child type, the child is **unreachable**:

```
1  try {
2      int[] arr = new int[3];
3      arr[5] = 10;
4  } catch (Exception e) {           // catches EVERYTHING
5      System.out.println("Generic handler");
6  } catch (ArrayIndexOutOfBoundsException e) { // UNREACHABLE - compile error!
7      System.out.println("Specific handler");
8  }
9
10 // CORRECT ORDER: most specific first
11 try {
12     int[] arr = new int[3];
13     arr[5] = 10;
14 } catch (ArrayIndexOutOfBoundsException e) {
15     System.out.println("Specific: " + e.getMessage());
16 } catch (Exception e) {
17     System.out.println("Generic: " + e.getMessage());
18 }
```

JAVA

Specific: Index 5 out of bounds for length 3

OUTPUT

Multi-Catch (Java 7+)

Handle multiple exception types with one block using `|`:

```
1  try {
2      String s = "hello";
3      int num = Integer.parseInt(s);
4  } catch (NumberFormatException | ArithmeticException e) {
5      System.out.println("Math error: " + e.getMessage());
6  }
7  // Both NumberFormatException and ArithmeticException are caught here
```

NOTE

In a multi-catch, the variable `e` is implicitly `final` — you cannot reassign it inside the block. If you need to handle each type differently, use separate catch blocks.

Common Exception Quick Reference

EXCEPTION	TYPE	COMMON CAUSE
<code>NullPointerException</code>	Unchecked	Calling a method on <code>null</code>
<code>ArrayIndexOutOfBoundsException</code>	Unchecked	Accessing invalid array index
<code>StringIndexOutOfBoundsException</code>	Unchecked	Invalid index in <code>charAt()</code> or <code>substring()</code>
<code>NumberFormatException</code>	Unchecked	<code>Integer.parseInt("abc")</code>
<code>ArithmeticException</code>	Unchecked	Division by zero, invalid modulus
<code>ClassCastException</code>	Unchecked	Unsafe downcast
<code>IllegalArgumentException</code>	Unchecked	Invalid argument passed to a method
<code>StackOverflowError</code>	Error	Infinite recursion
<code>FileNotFoundException</code>	Checked	File does not exist
<code>IOException</code>	Checked	General I/O failure

throw — Manually Trigger an Exception

```

1  public void setAge(int age) {
2      if (age < 0 || age > 150) {
3          throw new IllegalArgumentException("Age must be between 0 and 150, got: " + age);
4      }
5      this.age = age;
6  }
7
8  // When called with invalid input:
9  // setAge(-5);
10 // Exception in thread "main" java.lang.IllegalArgumentException:
11 // Age must be between 0 and 150, got: -5

```

JAVA

throws — Declare Checked Exceptions

When a method might throw a **checked** exception, you must either catch it or declare it with `throws`:

```

1  import java.io.File;
2  import java.io.FileNotFoundException;
3  import java.util.Scanner;
4
5  // Declares that this method MIGHT throw FileNotFoundException
6  public Scanner readFile(String path) throws FileNotFoundException {
7      File file = new File(path);
8      return new Scanner(file);
9  }
10
11 // Caller must handle it:
12 public static void main(String[] args) {
13     try {
14         Scanner sc = readFile("missing.txt");
15     } catch (FileNotFoundException e) {
16         System.out.println("File not found: " + e.getMessage());
17     }
18 }

```

JAVA

NOTE

`throws` on a method is a **declaration** — it doesn't throw anything itself. It tells callers: "I might throw this — you need to handle it." You can list multiple exceptions: `throws IOException, ClassNotFoundException`.

Custom Exceptions

Create your own exception classes for domain-specific errors. They should extend `Exception` (for checked) or `RuntimeException` (for unchecked):


```

1 // Unchecked custom exception (most common for validation)
2 public class InvalidGradeException extends RuntimeException {
3     public InvalidGradeException(String message) {
4         super(message);
5     }
6 }
7
8 // Using it
9 public class Student {
10     private double gpa;
11
12     public void setGpa(double gpa) {
13         if (gpa < 0 || gpa > 4.0) {
14             throw new InvalidGradeException(
15                 "Invalid GPA: " + gpa + ". Must be between 0.0 and 4.0"
16             );
17         }
18         this.gpa = gpa;
19     }
20 }

```

```

1 Student s = new Student();
2 s.setGpa(4.5);
3 // Exception in thread "main" InvalidGradeException:
4 // Invalid GPA: 4.5. Must be between 0.0 and 4.0

```

Checked Custom Exception (Less Common)

```

1 // Checked – forces callers to handle it
2 public class DatabaseException extends Exception {
3     public DatabaseException(String message) {
4         super(message);
5     }
6     public DatabaseException(String message, Throwable cause) {
7         super(message, cause);
8     }
9 }
10
11 public void connect() throws DatabaseException {
12     // ... if connection fails:
13     throw new DatabaseException("Could not connect to database", sqlException);
14 }

```

QUICK REFERENCE — THROW VS THROWS

- › `throw` — a **statement** that actually creates and throws an exception object inside a method
- › `throws` — a **keyword on the method signature** that declares which checked exceptions it might throw
- › One `throw` per exception, but `throws` can list many: `throws A, B, C`
- › Unchecked exceptions: no need for `throws` (but you can add it for documentation)
- › Checked exceptions: `throws` is mandatory unless caught inside the method

A **try-with-resources** statement automatically closes any resource that implements **AutoCloseable** (Scanner, FileReader, FileWriter, Connection, etc.). No finally block needed.

Old Way vs New Way

```
1  // OLD WAY - verbose and error-prone
2  Scanner sc = null;
3  try {
4      sc = new Scanner(new File("data.txt"));
5      while (sc.hasNextLine()) {
6          System.out.println(sc.nextLine());
7      }
8  } catch (FileNotFoundException e) {
9      System.out.println("File not found");
10 } finally {
11     if (sc != null) sc.close(); // easy to forget!
12 }
13
14 // NEW WAY - clean and safe (Java 7+)
15 try (Scanner sc = new Scanner(new File("data.txt"))) {
16     while (sc.hasNextLine()) {
17         System.out.println(sc.nextLine());
18     }
19 } catch (FileNotFoundException e) {
20     System.out.println("File not found");
21 }
22 // sc is automatically closed here - no finally needed
```

JAVA

Multiple Resources

```
1  try (
2      BufferedReader reader = new BufferedReader(new FileReader("input.txt"));
3      BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"))
4  ) {
5      String line;
6      while ((line = reader.readLine()) != null) {
7          writer.write(line.toUpperCase());
8          writer.newLine();
9      }
10 } catch (IOException e) {
11     System.out.println("I/O error: " + e.getMessage());
12 }
13 // Both reader AND writer are automatically closed - in reverse order
```

JAVA

TIP

Always use try-with-resources for files, database connections, sockets, and streams. It guarantees cleanup even if an exception occurs or you use **return**. This is the standard practice in modern Java.

What Implements AutoCloseable?

Generics let you write classes and methods that work with **any type**, while the compiler enforces type safety at compile time.

The Problem Generics Solve

```
1 // WITHOUT generics - works, but no type safety
2 ArrayList list = new ArrayList();
3 list.add("Hello");
4 list.add(42);           // no compile error - any type accepted!
5 String s = (String) list.get(1); // ClassCastException at RUNTIME!
```

JAVA

Exception in thread "main" java.lang.ClassCastException: class java.lang.Integer cannot be cast to class java.lang.String

OUTPUT

```
1 // WITH generics - wrong type caught at COMPILE time
2 ArrayList<String> list = new ArrayList<>();
3 list.add("Hello");
4 list.add(42); // COMPILE ERROR - Integer is not a String!
```

JAVA

TIP The core idea: **catch errors at compile time, not at runtime**. A compile error is always better than a runtime exception — you see it immediately in your IDE, not at 3 AM in production.

Generic Type Parameter Naming Convention

LETTER	MEANING	EXAMPLE
T	Type (any type)	Box<T> , print(T item)
E	Element (used in collections)	ArrayList<E>
K	Key (used in maps)	HashMap<K, V>
V	Value (used in maps)	HashMap<K, V>
N	Number	Calculator<N extends Number>
R	Return type	Supplier<R>

Generic Class

```
1  public class Pair<T, U> {
2      private T first;
3      private U second;
4
5      public Pair(T first, U second) {
6          this.first = first;
7          this.second = second;
8      }
9
10     public T getFirst() { return first; }
11     public U getSecond() { return second; }
12
13     @Override
14     public String toString() {
15         return "(" + first + ", " + second + ")";
16     }
17 }
```

JAVA

```
1  Pair<String, Integer> nameAge = new Pair<>("Ali", 20);
2  Pair<String, String> nameEmail = new Pair<>("Ali", "ali@email.com");
3
4  System.out.println(nameAge);    // (Ali, 20)
5  System.out.println(nameEmail);  // (Ali, ali@email.com)
```

JAVA

```
(Ali, 20) (Ali, ali@email.com)
```

OUTPUT

Generic Method

```

1  public class Utils {
2      // Generic method – the <T> comes BEFORE the return type
3      public static <T> T getFirst(ArrayList<T> list) {
4          if (list == null || list.isEmpty()) {
5              return null;
6          }
7          return list.get(0);
8      }
9
10     // Generic method with multiple type parameters
11     public static <K, V> void printEntry(K key, V value) {
12         System.out.println(key + " = " + value);
13     }
14
15     // Generic method that works on arrays
16     public static <T> void printArray(T[] arr) {
17         for (T item : arr) {
18             System.out.print(item + " ");
19         }
20         System.out.println();
21     }
22 }

```

```

1  ArrayList<String> names = new ArrayList<>();
2  names.add("Ali"); names.add("Sara");
3  System.out.println(Utils.getFirst(names)); // "Ali"
4
5  Utils.printEntry("Age", 20); // Age = 20
6
7  String[] words = {"Hello", "World"};
8  Utils.printArray(words); // Hello World
9
10 Integer[] nums = {1, 2, 3};
11 Utils.printArray(nums); // 1 2 3

```

Ali Age = 20 Hello World 1 2 3

Generic Method — Type Inference

You usually don't need to specify the type explicitly — Java infers it from the arguments:

```

1  // Type specified explicitly
2  Utils.<String>getFirst(names);
3
4  // Type inferred automatically – this is cleaner and the standard way
5  Utils.getFirst(names);

```

The type parameter `<T>` on a generic method goes **before** the return type: `public static <T> T getFirst(...)`. This is different from a generic class, where it goes **after** the class name: `class Box<T>`.

6.8 Bounded Types & Wildcards

Bounded Type Parameters — extends

Use `<T extends SomeClass>` to restrict the type to a class or its subclasses:

```
1 public class NumberBox<T extends Number> {
2     private T value;
3
4     public NumberBox(T value) {
5         this.value = value;
6     }
7
8     // Works because all Number subclasses have doubleValue()
9     public double doubleValue() {
10         return value.doubleValue();
11     }
12 }
13
14 NumberBox<Integer> intBox = new NumberBox<>(42);
15 NumberBox<Double> dblBox = new NumberBox<>(3.14);
16 NumberBox<String> strBox = new NumberBox<>("hi"); // COMPILER ERROR
17 // String does not extend Number
```

JAVA

Multiple bounds use `&`:

```
1 public static <T extends Number & Comparable<T>> T findMax(T a, T b) {
2     return (a.compareTo(b) ≥ 0) ? a : b;
3 }
4
5 System.out.println(findMax(3, 7)); // 7
6 System.out.println(findMax(2.5, 1.1)); // 2.5
```

JAVA

MISTAKE

You can only use **one class** in the extends clause, but multiple interfaces: `<T extends Number & Comparable<T> & Serializable>` is valid. `<T extends Number & String>` is invalid — two classes.

Wildcards — ?

A wildcard `?` means "any type". There are three forms:

Unbounded Wildcard: `?`


```

1 public static void printList(ArrayList<?> list) {
2     for (Object item : list) {
3         System.out.println(item);
4     }
5 }
6
7 printList(new ArrayList<String>()); // OK
8 printList(new ArrayList<Integer>()); // OK

```

Upper-Bounded Wildcard: ? extends T

```

1 // Accepts List of T or any subtype of T
2 public static double sum(ArrayList<? extends Number> list) {
3     double total = 0;
4     for (Number n : list) {
5         total += n.doubleValue();
6     }
7     return total;
8 }
9
10 sum(new ArrayList<Integer>()); // OK - Integer extends Number
11 sum(new ArrayList<Double>()); // OK - Double extends Number
12 sum(new ArrayList<String>()); // COMPILER ERROR - String does not extend Number

```

Lower-Bounded Wildcard: ? super T

```

1 // Accepts List of T or any supertype of T
2 // Useful when you want to ADD items to the list
3 public static void addIntegers(ArrayList<? super Integer> list) {
4     list.add(1);
5     list.add(2);
6 }
7
8 addIntegers(new ArrayList<Integer>()); // OK
9 addIntegers(new ArrayList<Number>()); // OK - Number is a supertype of Integer
10 addIntegers(new ArrayList<Object>()); // OK - Object is a supertype of Integer
11 addIntegers(new ArrayList<String>()); // COMPILER ERROR

```

WARNING

The rule of thumb: use `? extends T` when you **read** from a collection, and `? super T` when you **write** to it. If you use `? extends Number` and call `list.add()`, it won't compile — the compiler can't guarantee the added element is the right type.

1. Catching Exception Instead of the Specific Type

```
1 // BAD – too broad, hides the real problem
2 try {
3     int[] arr = new int[3];
4     arr[5] = 10;
5 } catch (Exception e) {
6     System.out.println("Something went wrong"); // what went wrong?!
7 }
8
9 // GOOD – specific catch tells you exactly what happened
10 try {
11     int[] arr = new int[3];
12     arr[5] = 10;
13 } catch (ArrayIndexOutOfBoundsException e) {
14     System.out.println("Array index error: " + e.getMessage());
15 }
```

JAVA

2. Empty catch Block — Swallowing Exceptions

```
1 // VERY BAD – the exception silently disappears
2 try {
3     int result = 10 / 0;
4 } catch (ArithmeticException e) {
5     // empty – nobody knows this failed!
6 }
7
8 // GOOD – at minimum, log it
9 try {
10     int result = 10 / 0;
11 } catch (ArithmeticException e) {
12     System.err.println("Error: " + e.getMessage());
13     e.printStackTrace(); // prints full stack trace
14 }
```

JAVA

3. Using Generics with Primitive Types

```
1 // WRONG
2 ArrayList<int> list = new ArrayList<int>();
3
4 // CORRECT – use the wrapper class
5 ArrayList<Integer> list = new ArrayList<Integer>();
6 // Autoboxing handles int → Integer for you
```

JAVA

4. Cannot Create Generic Arrays Directly

```

1  // WRONG – generic array creation is not allowed
2  T[] arr = new T[10];
3
4  // WORKAROUND – use Object array and cast
5  Object[] arr = new Object[10];
6  arr[0] = "Hello";
7  arr[1] = 42;
8  String s = (String) arr[0]; // OK
9  // Integer n = (Integer) arr[1]; // OK but risky – no compile-time safety

```

5. Confusing instanceof with Generics

```

1  // Generics use type erasure – at runtime, ArrayList<String> and ArrayList<Integer>
2  // are BOTH just ArrayList. instanceof cannot check generic type parameters:
3
4  ArrayList<String> strings = new ArrayList<>();
5  System.out.println(strings instanceof ArrayList<String>); // COMPILE ERROR
6
7  // This works (checking the raw type):
8  System.out.println(strings instanceof ArrayList); // true

```

6. Forgetting throws on Checked Exceptions

```

1  // If readFile declares "throws IOException":
2  public void process() {
3      Scanner sc = readFile("data.txt"); // COMPILE ERROR – unhandled IOException
4
5      // Fix option 1: catch it
6      try {
7          Scanner sc = readFile("data.txt");
8      } catch (IOException e) {
9          System.out.println("Error: " + e.getMessage());
10     }
11
12     // Fix option 2: declare it
13     public void process() throws IOException {
14         Scanner sc = readFile("data.txt");
15     }
16 }

```

7. Catching Exception When You Shouldn't

```

1  // BAD - catching and ignoring a fatal error
2  try {
3      connectToDatabase();
4  } catch (Exception e) {
5      // silently swallowed - database is broken, app continues in broken state
6  }
7
8  // BETTER - let it propagate to a level that can actually handle it
9  public void connectToDatabase() throws DatabaseException {
10     // if connection fails, throw - let the caller decide what to do
11 }

```

QUICK REFERENCE - EXCEPTION HANDLING DECISIONS

- › Catch the **most specific** exception type — don't catch `Exception` when `NullPointerException` is what you expect
- › Only catch exceptions you can **meaningfully handle**. If you can't, let them propagate up.
- › Use `e.printStackTrace()` during debugging; use `e.getMessage()` in production code.
- › Use **try-with-resources** for all resources — always.
- › Never leave a catch block empty — at minimum, log the error.
- › Use custom exceptions for domain-specific validation — they make error messages clear.

QUICK REFERENCE - GENERICS DECISIONS

- › Use generics **always** for collections — never use raw types
- › Prefer specific types (`ArrayList<String>`) over wildcards — add wildcards only when needed
- › Use `? extends T` when you only **read** from a collection
- › Use `? super T` when you need to **write** to a collection
- › Use `<T extends Number>` when your generic code calls Number methods
- › Remember: generics are **compile-time only**. At runtime, `ArrayList<String>` is just `ArrayList` (type erasure)

This document was compiled from the following resources. All content was adapted and restructured for educational use.

[1] **Programiz — Java Exception Handling**

try-catch-finally, throw/throws, custom exceptions, try-with-resources

<https://www.programiz.com/java-programming/exception-handling>

[2] **Programiz — Java Generics**

Generic classes, methods, bounded types, wildcards

<https://www.programiz.com/java-programming/generics>

[3] **GeeksforGeeks — Exception Handling in Java**

Exception hierarchy, checked vs unchecked, best practices, common pitfalls

<https://www.geeksforgeeks.org/exception-handling-java/>

[4] **GeeksforGeeks — Java Generics — Type parameters, wildcards, bounded types, type erasure**

<https://www.geeksforgeeks.org/generics-in-java/>

[5] **Oracle — Java Tutorials — Exceptions**

The catch-or-declare requirement, try-with-resources, creating exception classes

<https://docs.oracle.com/javase/tutorial/essential/exceptions/>

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com